Développer un algorithme de recherche en JavaScript

Projet Les petits plats

Sonia KECHIT

Sommaire

1 Présentation du contexte

2 Présentation du projet

3 Fonctionnalités

4 Choix et contraintes techniques

? Fiche d’investigation

5 Démonstration

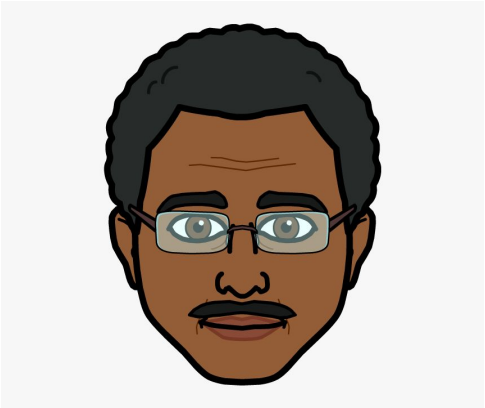
6 Bilan

7 Questions

Présentation du contexte

L'ÉQUIPE

Sandra
Cheffe de projet



Jean-Baptiste
Lead dev

Sonia
Développeuse front-end

Présentation du projet

Après avoir édité des livres de cuisine pendant plusieurs années, l'entreprise "Les petits plats" a décidé de se lancer dans un nouveau projet : réaliser son propre site de recettes de cuisine à l'instar de Marmiton ou 750g.

L'entreprise veut mettre l'accent sur:

- La fluidité du moteur de recherche
- L'utilisation d'algorithmes différents pour comparer leurs performances
- L'importance de la création d'une fiche d'investigation de fonctionnalités (document de comparaison) afin de choisir l'algorithme le plus performant

Fonctionnalités

1. La recherche doit pouvoir se faire via le champ principal ou via les tags (ingrédients, ustensiles ou appareil)
2. La recherche principale se lance à partir de 3 caractères entrés par l'utilisateur dans la barre de recherche
3. La recherche s'actualise pour chaque nouveau caractère entré
4. La recherche principale affiche les premiers résultats le plus rapidement possible
5. Les champs ingrédients, ustensiles et appareil de la recherche avancée proposent seulement les éléments restant dans les recettes présentes sur la page
6. Les retours de recherche doivent être une intersection des résultats. Si l'on ajoute les tags "coco" et "chocolat" dans les ingrédients, on doit récupérer les recettes qui ont à la fois de la coco et du chocolat.

Choix et contraintes techniques

Choix techniques

- Html5
- Javascript
- Sass

Contraintes techniques

- Fiche d'investigation de fonctionnalité sur l'algorithme de recherche (format PDF). Y intégrer le choix de l'algorithme définitif en comparaison à l'autre algorithme de recherche développé,
- Un fichier au format TXT contenant le lien vers votre code sur GitHub comprenant les deux branches avec les deux solutions différentes pour la recherche
- Aucune librairie ne sera utilisée pour le JavaScript du moteur de recherche,
- Utiliser un outil de comparaison de performance algorithmique
- Les couleurs de la charte: #D04F4F (logo), #3282F7, #68D9A4, #ED6454 (tags), #C7BEBE, #E7E7E7 (couleurs de fond),
- Polices du site: Roboto et Lato
- Validation W3C en HTML et CSS sans erreur

Fiche d’investigation

Comparatif

Boucles natives

Pour:

- Connues, appliquées et maîtrisées par un plus grand nombre de développeurs
- Facilité d’écriture

Contre:

- Impossibilité de leur coller les méthodes filter(), sort(), reduce()...
- Un code plus long, moins claire, plus difficilement maintenable
- Rapidité d'exécution légèrement plus faible

Programmation fonctionnelle

Pour:

- Méthode map() chainable
- Possibilité d’utiliser les méthodes filter(), sort(), reduce()...
- Code clair et concis
- Rapidité d'exécution du code
- Un code maintenable

Contre:

- Technique plus récente

Démonstration

Bilan

- Création d'algorithmes de recherche
- Comparaison d’algorithmes à l’aide d’une fiche d’investigation
- Boucles natives VS programmation fonctionnelle
- Event (tag cliqué, input sélectionné = réaction à une action)
- Méthodes: filter(), reduce() / flat(), concat(), flatMap()
- Comparaison de valeurs simplifiée
- Tableaux (agir dessus et réutiliser le(s) résultat(s))

Questions ?

Les petits plats